

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій
Кафедра радіоелектронних і комп'ютерних систем

КУРСОВА РОБОТА

на тему:

“Розробка веб-застосунку для візуалізації просторових даних на інтерактивній карті”

Студента 5 курсу, групи ФЕІм-14,
напряму підготовки «122
Комп'ютерні науки»
Теплий Д. І.

Керівник: доцент кафедри радіоелектронних і комп'ютерних систем
Соколовський Б.С.

Національна шкала _____

Кількість балів: _____

Оцінка: ECTS _____

Анотація

У даній курсовій роботі розглянуто процес розробки веб-застосунку для відображення геопросторових даних на інтерактивній карті. У роботі проаналізовано особливості геопросторових даних, формати їх представлення та сучасні технології веб-розробки, які використовуються для створення картографічних систем.

Розроблений веб-застосунок підтримує роботу з форматами WKT та GeoJSON, забезпечує конвертацію геоданих та їх подальше відображення на інтерактивній карті за допомогою бібліотеки Leaflet. Серверна частина реалізована на основі Java та Spring Framework. Додатково реалізовано підтримку роботи з шарами карти, зміну базової картографічної підкладки, а також обчислення довжини ліній та площі полігонів.

У результаті виконання роботи створено функціональний веб-застосунок для візуалізації та базової обробки геопросторових даних.

Abstract

This course work considers the development process of a web application for displaying geospatial data on an interactive map. The paper analyzes the features of geospatial data, data representation formats, and modern web development technologies used for creating cartographic systems.

The developed web application supports WKT and GeoJSON formats, provides geodata conversion, and displays the results on an interactive map using the Leaflet library. The server side is implemented using Java and Spring Framework. Additional functionality includes layer management, changing the map background, and calculating line lengths and polygon areas.

As a result of the work, a functional web application for visualization and basic processing of geospatial data was developed.

Зміст

Вступ	4
Постановка задачі	5
Розділ 1. Аналіз предметної області	6
1.1. Розробка веб-додатку	6
1.2. Розробка серверної частини веб-додатку	6
1.3. Комунікація між сервісами	7
1.4. Геопросторові дані та особливості їх розробки	8
1.5. Формати представлення геоданих	10
1.5.1. Формат WKT	10
1.5.2. Формат GeoJSON	11
1.6. Аналіз існуючих аналогів	12
Розділ 2. Інструменти розробки	14
2.1. Java	14
2.1.1. Область застосування	14
2.1.2. Структура програми на Java	16
2.1.3. Збірка проєкту на Java	16
2.2. Spring	19
2.2.1. Основні особливості	19
2.2.2. Чому Spring?	20
2.3. Leaflet	21
2.3.1. Основні можливості бібліотеки	21
2.3.2. Робота з інтерактивною картою	22
2.3.3. Відображення геометричних об'єктів	22
2.4. HTML, CSS та JavaScript	23
2.4.1. HTML для структури сторінки	23
2.4.2. CSS для стилізації інтерфейсу	23
2.4.3. JavaScript для взаємодії з картою	24

Розділ 3. Опис програмної реалізації.....	24
3.1. Архітектура серверної частини	25
3.2. Структура проєкту	26
3.2. Реалізація інтерактивної карти	27
Розділ 4. Функціонал проєкту.....	28
4.1. Головна сторінка	28
4.2. Введення та обробка геоданих	29
4.3. Відображення об'єктів на карті	30
4.3. Робота з шарами та зміна фону карти	32
Висновок	34
Список використаних джерел.....	35

Вступ

У сучасному світі геопросторові дані відіграють важливу роль у багатьох сферах діяльності людини. Вони використовуються у навігаційних системах, логістиці, геоінформаційних системах, транспортній інфраструктурі, міському плануванні, екологічному моніторингу та багатьох інших галузях. Завдяки розвитку веб-технологій з'явилась можливість створювати інтерактивні картографічні застосунки, які дозволяють ефективно візуалізувати, аналізувати та обробляти геопросторову інформацію без необхідності використання спеціалізованого програмного забезпечення.

Одним із важливих аспектів роботи з геоданими є підтримка різних форматів їх представлення. Найбільш поширеними форматами є WKT (Well-Known Text) та GeoJSON, які дозволяють описувати геометричні об'єкти, такі як точки, лінії та полігони. Для коректного відображення

таких даних у веб-застосунках необхідно використовувати сучасні бібліотеки та технології, що забезпечують інтерактивну взаємодію користувача з картою.

Актуальність даної курсової роботи полягає у необхідності створення зручного веб-застосунку для візуалізації геопросторових даних на інтерактивній карті. Такий застосунок повинен забезпечувати підтримку різних форматів геоданих, можливість їх обробки та відображення, а також додатковий функціонал для взаємодії з картою.

Метою курсової роботи є розробка веб-застосунку для відображення геопросторових даних на інтерактивній карті з використанням сучасних веб-технологій та серверної частини на основі Java і Spring Framework.

Постановка задачі

Для досягнення поставленої мети необхідно виконати такі завдання:

- проаналізувати особливості геопросторових даних та форматів їх представлення;
- дослідити сучасні технології для розробки веб-застосунків;
- реалізувати серверну частину для обробки та конвертації геоданих;
- створити інтерактивну карту з підтримкою відображення геометричних об'єктів;
- реалізувати функції зміни шарів карти та масштабування;
- додати можливість обчислення довжини ліній та площі полігонів.

Розділ 1. Аналіз предметної області

1.1. Розробка веб-додатку

Веб-додаток – це програмне забезпечення або програма, яку можна відкрити за допомогою будь-якого браузера. Зовнішній інтерфейс веб програми розробляється за допомогою таких мов програмування: HTML, CSS, Javascript, які підтримуються на будь-якому браузері (Opera, Chrome, Mozilla, Yandex). У той час як для написання серверної частини (Back-end) може використовуватися будь-яка інша мова програмування або фреймворк, Python, PHP, Ruby, Java. [1]

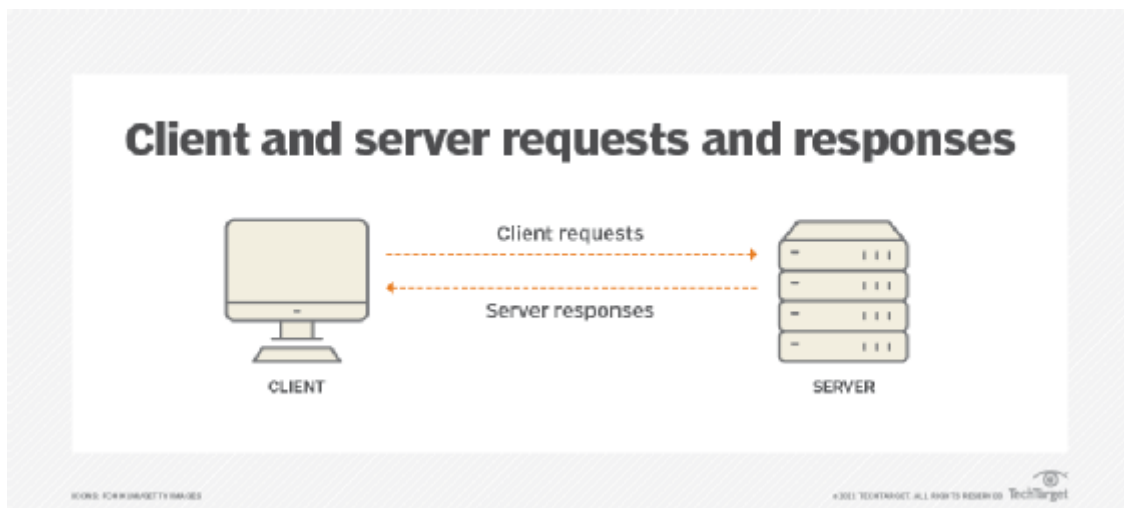


Рис. 1.1 Принцип роботи веб-застосунків

Зазвичай програмісти використовують веб-фреймворки для написання свого коду. Ці фреймворки містять набір функцій, об'єктів, правил та інших конструкцій, які спрощують розв'язання загальних проблем, прискорюють процес розробки та спрощують вирішення різних завдань у певній області.

1.2. Розробка серверної частини веб-додатку

Серверна частина - це програмний код, який працює на сервері і відповідає за створення вмісту веб-сторінок. Вона включає код для створення бази даних, сервера та додатка, а також відповідає за міграцію бази даних,

інтеграцію API та налаштування серверних технологій для забезпечення роботи сайту.[5][2]

Я вважаю, що головною перевагою серверної розробки є можливість персоналізації вмісту сайту для окремих користувачів. Динамічні сайти можуть відображати більш релевантний вміст, орієнтуючись на уподобання та звички користувачів. Це також спрощує використання сайтів, дозволяючи зберігати особисті налаштування та інформацію, як-от збережені дані кредитних карток, для подальшого використання та оптимізації процесу платежів.

1.3. Комунікація між сервісами

Є кілька способів комунікації між сервісами:

- REST
- SOAP
- GraphQL

Ці підходи до архітектури мережових протоколів надають доступ до інформаційних ресурсів. Який з них оптимальний? Кожен підхід має свої переваги та недоліки.

SOAP використовує стандарт XML для комунікації, що може призвести до надмірного обсягу передаваних даних і погіршення продуктивності програми. Однак SOAP забезпечує високий рівень безпеки та надійності, що може бути важливим для корпоративних застосунків.

GraphQL, як новіший підхід, дозволяє клієнтам запитувати саме ті дані, які їм потрібні, що зменшує обсяг передаваних даних. Водночас, він може бути складним у реалізації запитів і не має певних важливих функцій, таких як механізм кешування запитів і лімітування кількості запитів за певний проміжок часу. Тим не

менш, його гнучкість та ефективність у деяких випадках перевершують інші методи.

REST є найпоширенішим підходом і базується на стандарті HTTP. Він дозволяє швидко та ефективно будувати сервіси та передавати дані мережею без зайвого навантаження на неї. Проте REST може стикатися з проблемами надмірного або недостатнього отримання даних із запиту та слабкою типізацією, що може вплинути на спілкування між програмами.

Незважаючи на деякі обмеження, REST API виявився найбільш зручним та ефективним для реалізації багатьох застосунків. Його простота та сумісність з багатьма клієнтськими інтерфейсами роблять його популярним вибором для розробників, забезпечуючи швидку передачу даних і легке налаштування сервісів.

Крім того, REST API добре підтримується більшістю сучасних мов програмування та має велику спільноту, що сприяє легкому вирішенню проблем і швидкому освоєнню нових розробників.

What is Rest API?

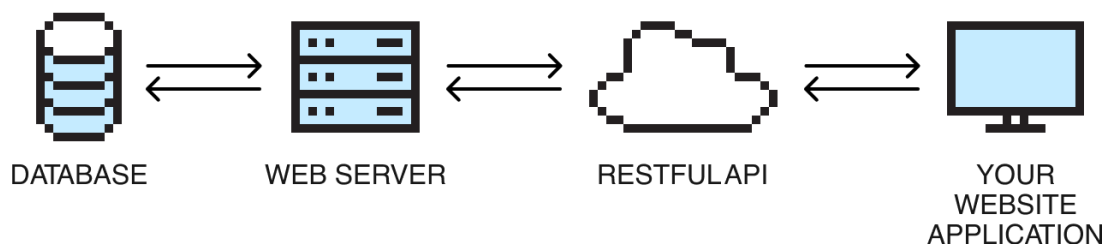


Рис. 1.2 REST API. Приклад комунікації на рівні сервісів

1.4. Геопросторові дані та особливості їх розробки

Геопросторові дані — це інформація, яка має прив’язку до конкретного місця на поверхні Землі. Вони описують об’єкти реального світу через їх

геометричні характеристики та координати, що дозволяє виконувати їх візуалізацію, аналіз та подальшу обробку в геоінформаційних системах (ГІС) та веб-застосунках.

Основними типами геометричних об'єктів у геопросторових даних є точки (Point), лінії (LineString) та полігони (Polygon). Точкові об'єкти використовуються для позначення конкретних місць, наприклад, будівель або координат подій. Лінійні об'єкти застосовуються для представлення доріг, маршрутів або інших протяжних елементів. Полігони використовуються для опису площ об'єктів, таких як земельні ділянки, будівлі або адміністративні межі.

Особливістю роботи з геопросторовими даними є необхідність їх точного зберігання, передачі та відображення. Для цього використовуються спеціальні формати даних, які забезпечують стандартизований опис геометрії. Найбільш поширеними з них є WKT (Well-Known Text) та GeoJSON.

Додатковою складністю є необхідність врахування систем координат та можливих трансформацій між ними, оскільки різні джерела даних можуть використовувати різні просторові системи відліку. Це вимагає коректної обробки даних на стороні серверної частини та їх адаптації для відображення у веб-картографічних бібліотеках.

У сучасних веб-застосунках геопросторові дані зазвичай використовуються для побудови інтерактивних карт, які дозволяють користувачам взаємодіяти з об'єктами, виконувати вимірювання відстаней і площ, а також аналізувати просторову інформацію в реальному часі.

1.5. Формати представлення геоданих

1.5.1. Формат WKT

WKT (Well-Known Text) — це текстовий формат представлення геопросторових даних, який використовується для опису геометричних об'єктів у вигляді зрозумілого для людини рядка. Він був розроблений для стандартизованого обміну геоданими між різними геоінформаційними системами та програмними застосунками.

Формат WKT дозволяє описувати основні типи геометрії, такі як точки, лінії та полігони. Наприклад, точка задається координатами у вигляді POINT(longitude latitude), лінія — як послідовність координат LINESTRING(...), а полігон — як замкнена послідовність точок POLYGON(...). Завдяки своїй простоті та компактності, WKT широко використовується для передачі геометричних даних між серверною та клієнтською частинами веб-застосунків.

- Приклад запису точки у форматі WKT:

```
POINT(30.5238 50.4501)
```

- Приклад лінії:

```
LINESTRING(30.5 50.4, 31.0 50.5, 31.5 50.6)
```

- Приклад полігону:

```
POLYGON((30 50, 31 50, 31 51, 30 51, 30 50))
```

У контексті розробленого веб-застосунку WKT використовується як один із вхідних форматів для введення геометричних даних користувачем. На серверній стороні ці дані парсяться та конвертуються у внутрішнє представлення або у формат GeoJSON для подальшого відображення на інтерактивній карті.

1.5.2. Формат GeoJSON

GeoJSON — це відкритий формат обміну геопросторовими даними, який базується на JSON (JavaScript Object Notation). Він використовується для представлення просторових об'єктів разом з їх атрибутами у структурованому та зручному для веб-застосунків вигляді.

Основною перевагою GeoJSON є його сумісність із сучасними веб-технологіями, зокрема JavaScript, що робить його особливо зручним для використання в інтерактивних картографічних застосунках. Формат підтримує основні типи геометрії: точки (Point), лінії (LineString), полігони (Polygon), а також їх колекції (FeatureCollection).

Структура GeoJSON базується на об'єктах типу Feature, які містять геометрію та додаткові властивості (properties), що дозволяє зберігати не лише координати, але й супровідну інформацію про об'єкти.

- Приклад точки у форматі GeoJSON:

```
{  
  "type": "Point",  
  "coordinates": [30.5238, 50.4501]  
}
```

- Приклад лінії:

```
{  
  "type": "LineString",  
  "coordinates": [  
    [30.5, 50.4],  
    [31.0, 50.5],  
    [31.5, 50.6]  
  ]  
}
```

- Приклад полігону:

```
{  
  "type": "Polygon",  
  "coordinates": [  
    [  
      [30, 50],  
      [31, 50],  
      [31, 51],  
      [30, 51],  
      [30, 50]  
    ]  
  ]  
}
```

У розробленому веб-застосунку GeoJSON використовується як один із основних форматів передачі геопросторових даних між клієнтською та серверною частинами. Після введення користувачем даних або у форматі WKT, вони можуть бути конвертовані у GeoJSON для подальшого відображення на інтерактивній карті за допомогою бібліотеки Leaflet.

1.6. Аналіз існуючих аналогів

Існує ряд програмних засобів для роботи з геопросторовими даними та їх візуалізації. Одним із таких інструментів є JTS TestBuilder, який використовується для тестування та роботи з геометричними об'єктами у форматі WKT.

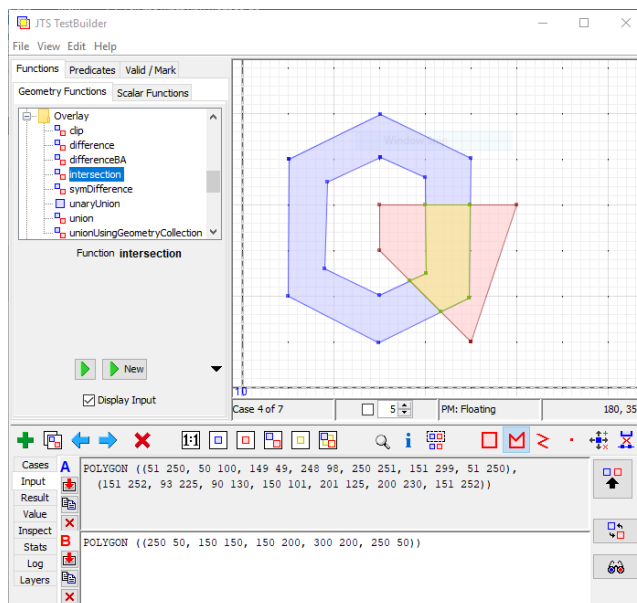


Рис. 1.6.1. Приклад роботи JTS TestBuilder

Даний застосунок дозволяє створювати та відображати геометричні об'єкти, виконувати базові операції над ними та аналізувати результати обробки геометрії. Проте він має ряд недоліків у контексті сучасних веб-застосунків.

Основним недоліком є те, що JTS TestBuilder є десктопною програмою, яка потребує локального встановлення та працює лише у вигляді окремого застосунку для операційної системи. Інтерфейс програми є застарілим та менш зручним для користувача порівняно із сучасними веб-рішеннями.

Крім цього, застосунок орієнтований переважно на роботу з форматом WKT та не забезпечує повноцінної підтримки сучасних веб-форматів, таких як GeoJSON, який широко використовується у веб-картографії та JavaScript-застосунках.

На відміну від існуючих аналогів, розроблений веб-застосунок:

- працює безпосередньо у браузері;
- не потребує встановлення додаткового програмного забезпечення;
- підтримує як WKT, так і GeoJSON;
- забезпечує інтерактивну роботу з картою;
- підтримує роботу з шарами та зміною базових карт;
- дозволяє обчислювати довжину ліній та площу полігонів.

Таким чином, розроблений застосунок є більш зручним та адаптованим до сучасних вимог веб-картографії.

Розділ 2. Інструменти розробки

2.1. Java

Java - це високорівнева об'єктно-орієнтована мова програмування, розроблена компанією Sun Microsystems (нині належить Oracle Corporation) у 1995 році. Вона широко використовується для створення веб-додатків, корпоративних систем, мобільних додатків та багатьох інших типів програмного забезпечення.[8]

2.1.1. Область застосування

Веб-розробка:

- Серверна частина (Back-end): Java широко використовується для створення серверних додатків, особливо у великих корпоративних системах. Популярні фреймворки, такі як Spring і JavaServer Faces (JSF), допомагають розробляти масштабовані та надійні веб-додатки.

Сервлети та JSP:

- Технології сервлетів та JavaServer Pages (JSP) дозволяють створювати динамічні веб-сторінки, які інтерактивно взаємодіють з користувачами.

Мобільні додатки:

- Android-розробка: Java є основною мовою програмування для створення додатків на платформі Android. Хоча Kotlin також стає популярним, Java залишається основою для багатьох Android-додатків.

Розподілені системи:

- Корпоративні рішення (Enterprise Solutions): Java часто використовується для створення масштабованих корпоративних додатків. Технології, такі як Java EE (Enterprise Edition), забезпечують створення надійних, багатошарових розподілених систем.

Мікросервіси:

- Java добре підходить для архітектури мікросервісів, де додаток розбивається на невеликі, незалежні служби, які можуть розгортатися окремо.

Наукові дослідження та великі дані (Big Data):

- Java використовується у наукових дослідженнях для обробки великих обсягів даних. Платформи, такі як Apache Hadoop і Apache Spark, мають підтримку Java.

Інтернет речей (IoT):

- Java добре підходить для розробки програмного забезпечення для пристроїв IoT завдяки своїй кросплатформеності та надійності. Java ME (Micro Edition) спеціально розроблена для вбудованих систем і пристроїв з обмеженими ресурсами.

Фінансові послуги:

- Банки, біржі та інші фінансові установи використовують Java для розробки безпечних та продуктивних систем обробки транзакцій, управління портфелями та іншого фінансового програмного забезпечення.

Геймдев (Game Development):

- Хоча Java не є найпоширенішою мовою для створення AAA-ігор, вона використовується для розробки мобільних та незалежних ігор. Популярний ігровий рушій LibGDX побудований на Java.

Вбудовані системи:

- Java використовується для створення програмного забезпечення для вбудованих систем, таких як принтери, смарт-карти та інші пристрої з обмеженими ресурсами.

Бази даних:

- Java часто використовується для роботи з базами даних. JDBC (Java Database Connectivity) є стандартним API для підключення та виконання операцій з базами даних.

Завдяки своїй універсальності, кросплатформеності та надійності, Java залишається однією з найпопулярніших мов програмування, що використовується у багатьох галузях промисловості.

2.1.2. Структура програми на Java

Код програми записують, як правило, у текстові файли з розширенням `.java`. Редагувати такі файли можна будь-яким текстовим редактором та запускати в будь-якому інтегрованому середовищі розробки.

Для мене найкращим варіантом є IntelliJ IDEA Ultimate Edition, який є пристосований під веб-розробку. Багато сучасних редакторів мають ряд особливостей, які необхідні для розробника (підсвічування синтаксису, автодоповнення, автоматичні відступи).

2.1.3. Збірка проєкту на Java

Збірка проєкту на Java включає кілька ключових етапів, які перетворюють вихідний код у виконуване програмне забезпечення. Інструменти збірки допомагають автоматизувати цей процес, що включає компіляцію, тестування, пакування та розгортання. Три основні інструменти збірки для Java — це Apache Maven, Gradle та Apache Ant. У цьому огляді основну увагу приділено Maven.

Apache Maven

Огляд:

Apache Maven — це потужний інструмент для управління проектами та автоматизації процесу збірки. Він базується на концепції Project Object Model (POM) і використовує XML-файли для опису конфігурації проекту, управління залежностями та плагінами.[9]

- Переваги Maven:

1. Управління залежностями:

Maven автоматично завантажує необхідні залежності з центрального репозиторію Maven або інших конфігурованих репозиторіїв. Це дозволяє розробникам зосередитися на написанні коду, не турбуючись про пошук та інтеграцію бібліотек.

2. Стандартизована структура проекту:

Maven використовує стандартну структуру каталогів, що робить проекти більш зрозумілими та підтримуваними. Це спрощує перенесення проекту між різними розробниками або командами.

3. Плагінова архітектура:

Maven має безліч плагінів, які автоматизують різні завдання, такі як компіляція коду, запуск тестів, генерація документації, пакування артефактів та розгортання.

4. Цикл життя проекту:

Maven визначає стандартизований життєвий цикл проекту, що включає фази, такі як компіляція, тестування, збірка і розгортання. Це дозволяє автоматизувати весь процес збірки з мінімальними зусиллями.

5. Інтеграція з CI/CD:

Maven добре інтегрується з інструментами безперервної інтеграції та доставки (CI/CD), такими як Jenkins, Bamboo та інші, що спрощує автоматизацію розгортання і тестування.

Gradle

Огляд:

Gradle — це сучасний інструмент збірки, який поєднує в собі найкращі риси Maven та Ant. Він використовує DSL (Domain Specific Language) на основі Groovy або Kotlin для опису конфігурацій збірки.[9]

- Переваги Gradle:

1. Продуктивність:

Gradle оптимізований для швидкої збірки проектів завдяки використанню інкрементальних збірок, які компілюють лише ті частини проекту, які були змінені.

2. Гнучкість:

Використання DSL дозволяє налаштовувати процес збірки на високому рівні, що забезпечує більшу гнучкість у порівнянні з XML-конфігураціями Maven.

3. Управління залежностями:

Як і Maven, Gradle ефективно управляє залежностями проекту через репозиторії, такі як Maven Central та jCenter.

4. Інтеграція з інструментами:

Gradle добре інтегрується з популярними інструментами розробки, такими як Android Studio, що робить його основним вибором для Android-розробників.

Apache Ant

Огляд:

Apache Ant — це більш старий інструмент автоматизації збірки, який використовує XML-файли для опису завдань збірки. На відміну від Maven і Gradle, Ant не дотримується жорсткої структури проекту і не забезпечує автоматичного управління залежностями.[9]

- Переваги Ant:

1. Гнучкість:

Ant надає максимальну гнучкість у налаштуванні завдань збірки. Кожен аспект збірки може бути налаштований за допомогою XML-сценаріїв.

2. Розширюваність:

Ант має систему плагінів та бібліотек, що дозволяють розширювати його функціональність для специфічних потреб проекту.

3. Простота:

Ant простий у використанні для невеликих проектів і не вимагає складних налаштувань.

Для себе я обрав Maven, адже вважаю його найбільш перспективним та маю досвід роботи з ним .

2.2. Spring

Spring Framework є відкритим фреймворком з контейнерами, які підтримують інверсію управління для розробки додатків на платформі Java. Він має ключові особливості, які можуть бути використані в будь-якому додатку на Java, а також є розширення для створення веб-додатків на платформі Java EE. [10]

2.2.1. Основні особливості

Spring - це потужний фреймворк для розробки додатків на Java, який пропонує гнучкий та економний підхід до створення програмного забезпечення. Оскільки Spring є обширним, він поділяється на кілька модулів для кращої організації та ефективності.[10] Основні модулі включають:

- Spring Core:

Цей модуль є основою фреймворку Spring та містить контейнер IoC (Inversion of Control). IoC контейнер управляє створенням та керуванням об'єктами в програмі. Він дозволяє визначати та управляти залежностями між об'єктами безпосередньо у конфігураційних файлах.

- Spring AOP (Aspect-Oriented Programming):

Цей модуль дозволяє розділити програму на аспекти або проблеми, що дозволяє краще керувати перехресними функціями, такими як логування, транзакції та безпека.

- Spring MVC (Model-View-Controller):

Spring MVC надає реалізацію архітектури MVC для веб-додатків. Він розділяє додаток на модель (логіка додатку), вид (представлення) та контролер (логіка обробки запитів).

- Spring ORM (Object-Relational Mapping):

Цей модуль дозволяє спростити взаємодію з базами даних в програмі. Він надає шаблони та інструменти для роботи з ORM-рішеннями, такими як Hibernate та JPA.

2.2.2. Чому Spring?

Spring робить програмування на Java швидшим, простішим і безпечнішим для всіх. Фокус на швидкості, простоті та продуктивності зробив його найпопулярнішим у світі для розробки на Java.

- Spring всюди:

Розробники з усього світу довіряють гнучким бібліотекам Spring. Кожен день Spring дарує мільйонам кінцевих користувачів чудові враження.

- Spring гнучкий:

Завдяки гнучкому та розширюваному набору функцій і сторонніх бібліотек, Spring дозволяє розробникам створювати практично будь-яку програму. Він є основою для широкого спектру функцій і можливостей.

- Spring продуктивний:

Spring Boot перевершує ваш підхід до розробки на Java, роблячи його радикально простішим. З Spring Boot контекст програми та автоматично налаштований вебсервер доступні легко та швидко.[3]

- Spring безпечний:

З великим досвідом вирішення проблем безпеки, Spring гарантує надійність та безпеку вашого додатку.

2.3. Leaflet

Leaflet — це легка та популярна JavaScript-бібліотека для створення інтерактивних веб-карт. Вона широко використовується у веб-розробці завдяки простоті інтеграції, високій продуктивності та підтримці великої кількості картографічних сервісів і плагінів. Leaflet дозволяє швидко створювати динамічні карти з можливістю масштабування, переміщення та взаємодії з геопросторовими об'єктами.

2.3.1. Основні можливості бібліотеки

Leaflet надає набір інструментів для роботи з інтерактивними картами, зокрема:

- відображення тайлових шарів карти (OpenStreetMap, Mapbox та інші);
- підтримка маркерів, ліній, полігонів та інших геометричних об'єктів;
- можливість масштабування та переміщення карти;
- робота з шарами (layers) для гнучкого керування відображенням даних;
- підтримка поп-уп вікон та інтерактивних елементів;
- розширюваність через плагіни.

Завдяки цьому Leaflet є зручним інструментом для створення як простих картографічних інтерфейсів, так і більш складних геоінформаційних застосунків.

2.3.2. Робота з інтерактивною картою

Основою роботи з Leaflet є створення об'єкта карти та задання початкових параметрів, таких як центр карти та рівень масштабування. Після цього додається базовий шар тайлів, який формує візуальне представлення карти.

Користувач може взаємодіяти з картою шляхом:

- зміни масштабу (zoom in / zoom out);
- переміщення по карті (pan);
- додавання та видалення шарів;
- перегляду інформації про об'єкти.

У веб-застосунку інтерактивна карта використовується як основний інтерфейс для візуалізації геопросторових даних, що вводяться користувачем.

2.3.3. Відображення геометричних об'єктів

Leaflet підтримує відображення різних типів геометричних об'єктів, таких як точки, лінії та полігони. Ці об'єкти можуть бути створені як вручну, так і на основі даних у форматі GeoJSON.

- Точки зазвичай відображаються у вигляді маркерів.
- Лінії використовуються для представлення маршрутів або відрізків.
- Полігони застосовуються для відображення площ об'єктів.

Leaflet також дозволяє стилізувати геометричні об'єкти, змінюючи їх колір, товщину ліній, прозорість та інші параметри. Це забезпечує зручну візуальну диференціацію різних типів даних на карті.

У розробленому застосунку Leaflet використовується для відображення геопросторових об'єктів, отриманих після конвертації WKT або GeoJSON, а також для реалізації інтерактивної взаємодії користувача з картою.

2.4. HTML, CSS та JavaScript

2.4.1. HTML для структури сторінки

HTML (HyperText Markup Language) використовується як базова мова розмітки для створення структури веб-сторінки та визначення розташування основних елементів інтерфейсу. У даному веб-застосунку HTML відповідає за формування загальної композиції сторінки, включаючи поле для введення геопросторових даних у форматах WKT або GeoJSON, кнопки для ініціалізації обробки даних, а також контейнер, у якому відображається інтерактивна карта. Окремо також формуються додаткові блоки інтерфейсу, які відповідають за керування шарами карти та відображення результатів обчислень.

2.4.2. CSS для стилізації інтерфейсу

CSS (Cascading Style Sheets) застосовується для визначення візуального вигляду веб-застосунку та забезпечення зручності його використання. У межах даного проєкту CSS використовується для стилізації всіх основних елементів інтерфейсу, включаючи панель введення даних, кнопки взаємодії та блок відображення карти. Також за допомогою CSS реалізується правильне компонування елементів на сторінці, щоб карта займала основну частину екрану, а допоміжні елементи залишались доступними та не

перевантажували інтерфейс. Додатково стилі забезпечують адаптивність застосунку, що дозволяє коректно відображати інтерфейс на різних розмірах екранів.

2.4.3. JavaScript для взаємодії з картою

JavaScript є ключовою технологією, яка забезпечує інтерактивність веб-застосунку та взаємодію користувача з картою. У даному проєкті JavaScript використовується для обробки введених геопросторових даних, незалежно від того, чи це WKT, чи GeoJSON, після чого ці дані передаються на серверну частину через HTTP-запити для подальшої конвертації та обробки. Отримані результати повертаються назад і використовуються для відображення геометричних об'єктів на інтерактивній карті за допомогою бібліотеки Leaflet.

Крім цього, JavaScript відповідає за динамічне оновлення карти без перезавантаження сторінки, що дозволяє користувачу в реальному часі додавати нові об'єкти, змінювати їх відображення або видаляти їх. Також реалізовано взаємодію з шарами карти, включаючи перемикання базових картографічних підкладок та керування видимістю геометричних об'єктів. Додатково JavaScript використовується для виконання обчислень, таких як визначення довжини лінійних об'єктів та площі полігонів, що дозволяє розширити функціональність застосунку до рівня базового геоаналізу у веб-середовищі.

Розділ 3. Опис програмної реалізації

Даний розділ описує архітектуру створеного проєкту та його складових модулів

3.1. Архітектура серверної частини

Архітектура сервісної частини, коли використовується MVCS (Model-View-Controller-Service), спирається на чітке розділення обов'язків між різними компонентами системи.[4]

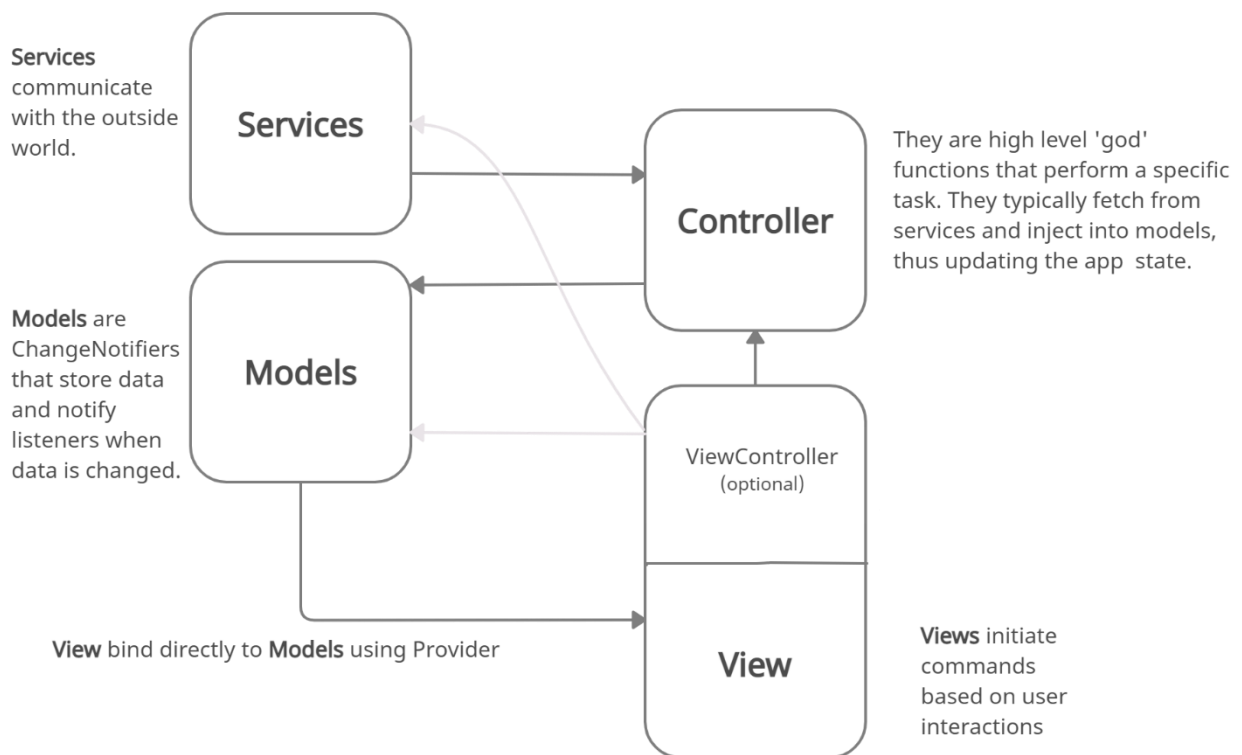
Модель (Model): Модель представляє собою структуру даних та логіку, яка опрацьовує ці дані. У контексті сервісної частини, модель відповідає за доступ до даних та їх обробку. Наприклад, це може бути шар бази даних або набір класів, які представляють об'єкти доменної моделі.

Вид (View): Вид відповідає за відображення інформації користувачу та взаємодію з ним. У контексті сервісної частини, вигляд може бути представлений як API або інтерфейс користувача, який надає доступ до функціональності системи.

Контролер (Controller): Контролер приймає запити від користувача або інших системних компонентів і керує обробкою цих запитів. У MVCS, контролер діє як посередник між моделлю та видом, викликаючи відповідні методи моделі для обробки запитів та ініціюючи оновлення відображення у вигляді.

Сервіс (Service): Додавши сервісний компонент, ми розділяємо бізнес-логіку від контролера. Сервіси містять логіку додатку, що не пов'язана з контролерами та не відноситься безпосередньо до обробки HTTP-запитів або інтерфейсів користувача. Сервіси можуть включати обробку даних, виконання бізнес-правил, зв'язок з базою даних тощо.

При використанні архітектури MVCS, контролер взаємодіє з моделлю та сервісом для обробки запитів, після чого відображає результат користувачеві через вид. Це забезпечує гнучкість та розділення обов'язків, що полегшує розробку, тестування та підтримку системи.



3.2. Структура проєкту

Структура серверної частини веб-застосунку побудована за типовим багат шаровим підходом, що дозволяє розділити відповідальність між різними компонентами системи та забезпечити простоту супроводу і масштабування коду. Проєкт реалізовано на основі Java із використанням Spring Framework, що дозволяє організувати логіку у вигляді окремих шарів.

У межах серверної частини виділено три основні пакети: `controller`, `service` та `dto`. Пакет `controller` відповідає за обробку HTTP-запитів від клієнтської частини та визначає точки входу в систему. Саме тут приймаються дані, які надходять у вигляді WKT або GeoJSON, після чого вони передаються далі для обробки.

Пакет `dto` використовується для передачі даних між клієнтською та серверною частинами, а також між внутрішніми компонентами системи. Це дозволяє уніфікувати структуру даних і відокремити внутрішню реалізацію від зовнішнього представлення.

Основна бізнес-логіка зосереджена у пакеті `service`, де виконується обробка геопросторових даних, їх конвертація між форматами та підготовка до подальшого використання. Такий підхід дозволяє ізолювати логіку обробки даних від рівня контролерів, що підвищує читабельність та підтримуваність коду.

Загалом така структура проєкту забезпечує чітке розділення відповідальностей між компонентами системи, що є важливим для розробки складніших веб-застосунків, які працюють із геопросторовими даними та потребують гнучкої архітектури.

3.2. Реалізація інтерактивної карти

Інтерактивна карта є одним із ключових компонентів розробленого веб-застосунку, оскільки саме вона забезпечує візуалізацію геопросторових даних та взаємодію користувача з ними. Для її реалізації використано бібліотеку `Leaflet`, яка дозволяє створювати динамічні карти з підтримкою різних типів шарів та геометричних об'єктів.

На стороні клієнта створюється базовий об'єкт карти, якому задається початковий центр координат та рівень масштабування. Далі підключається тайловий шар, що відповідає за підкладку карти, зокрема може використовуватися стандартний шар `OpenStreetMap` або інші доступні картографічні сервіси. Це дозволяє користувачу працювати з картою у звичному для нього вигляді та виконувати навігацію по території.

Основна функціональність інтерактивної карти полягає у відображенні геопросторових об'єктів, які надходять після обробки введених користувачем даних. Після конвертації WKT або GeoJSON у придатний для `Leaflet` формат,

відповідні геометрії додаються на карту у вигляді окремих шарів. Це можуть бути маркери для точок, ламані для ліній або полігони для площинних об'єктів.

Додатково реалізовано механізм керування шарами, що дозволяє користувачу вмикати або вимикати відображення окремих об'єктів, а також змінювати базову підкладку карти. Це підвищує зручність роботи з великою кількістю геоданих і дозволяє фокусуватися на потрібній інформації.

Окрему роль відіграє динамічна взаємодія з картою, яка реалізована без необхідності перезавантаження сторінки. Усі зміни, включаючи додавання нових об'єктів або зміну їх параметрів, відображаються миттєво, що забезпечує комфортний користувацький досвід при роботі з геопросторовими даними.

Розділ 4. Функціонал проекту

Даний розділ описує функціонал мого веб-застосунку.

4.1. Головна сторінка

Головна сторінка веб-застосунку реалізована як єдиний інтерфейс для взаємодії користувача з геопросторовими даними. Вона містить текстове поле для введення даних у форматі WKT або GeoJSON, випадаючий список для вибору формату введення, а також кнопку запуску обробки даних. Основну частину інтерфейсу займає інтерактивна карта, яка займає всю доступну праву частину екрану та використовується для візуалізації результатів.

Додатково на сторінці присутні елементи керування базовою картою, вибором одиниць вимірювання та список доданих шарів, що дозволяє користувачу керувати вже відображеними об'єктами.

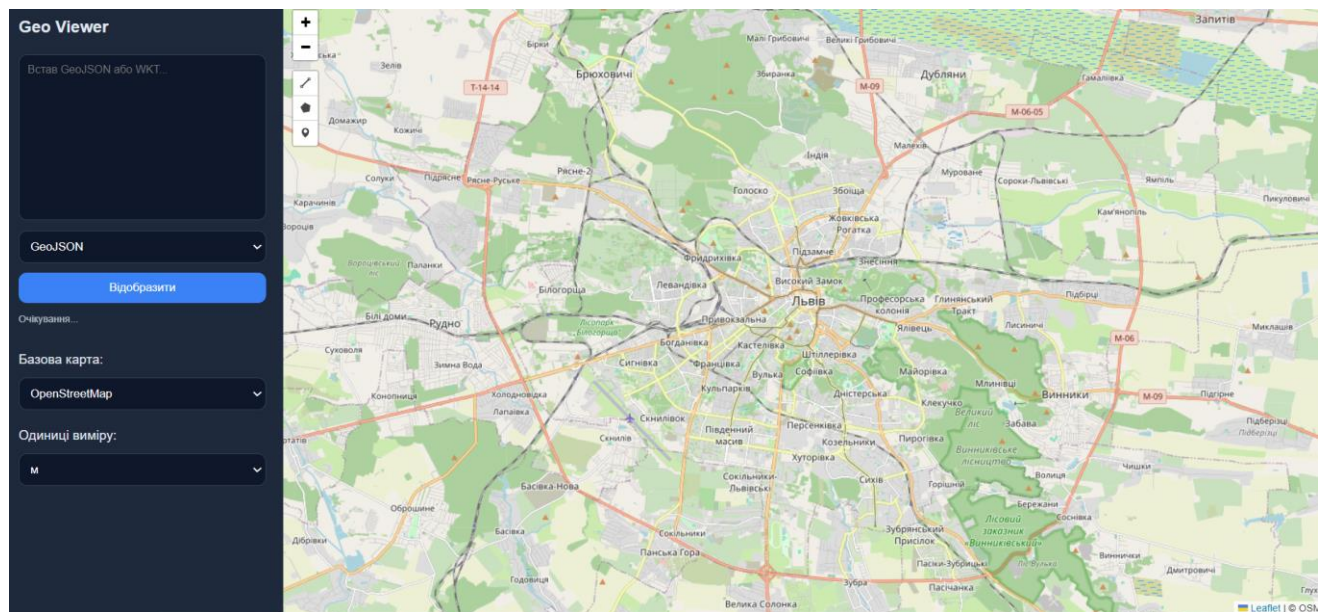
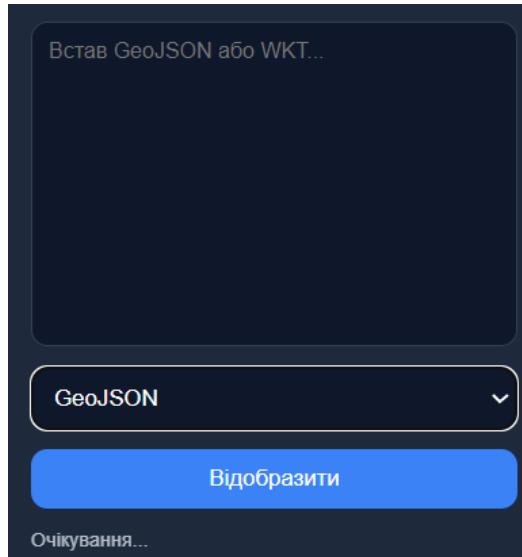


Рис. 4.1. Головна сторінка

4.2. Введення та обробка геоданих

Введення геопросторових даних здійснюється через текстове поле, у яке користувач може вставити як WKT, так і GeoJSON, після чого необхідно вибрати відповідний формат у випадаючому списку. Після натискання кнопки дані надсилаються на серверну частину через HTTP POST запит у форматі JSON.

Сервер виконує конвертацію вхідних даних у GeoJSON, який повертається на клієнтську сторону. У випадку помилки обробки користувач отримує відповідне повідомлення, тоді як у разі успішного виконання дані передаються для подальшого відображення на карті.



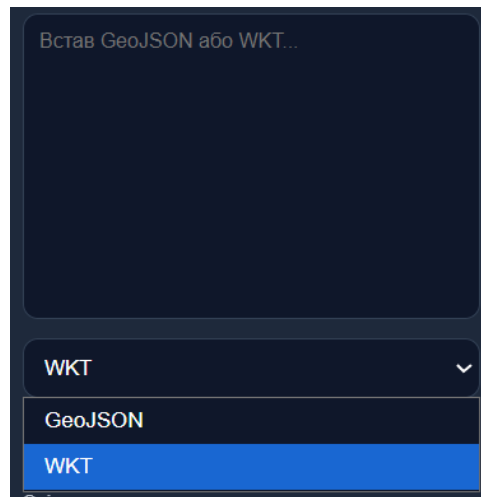
Встав GeoJSON або WKT...

GeoJSON

Відобразити

Очікування...

Рис. 4.2.1. Поле для вводу тексту



Встав GeoJSON або WKT...

WKT

GeoJSON

WKT

Очікування...

Рис. 4.2.2. Випадаючий список формату тексту

4.3. Відображення об'єктів на карті

Після отримання GeoJSON даних вони відображаються на інтерактивній карті за допомогою бібліотеки Leaflet. Для різних типів геометрії використовується різний спосіб рендерингу: точки відображаються у вигляді кругових маркерів, лінії та полігони — у вигляді векторних шарів.



Рис. 4.3.1. Вставка тексту формату GeoJSON та його відображення

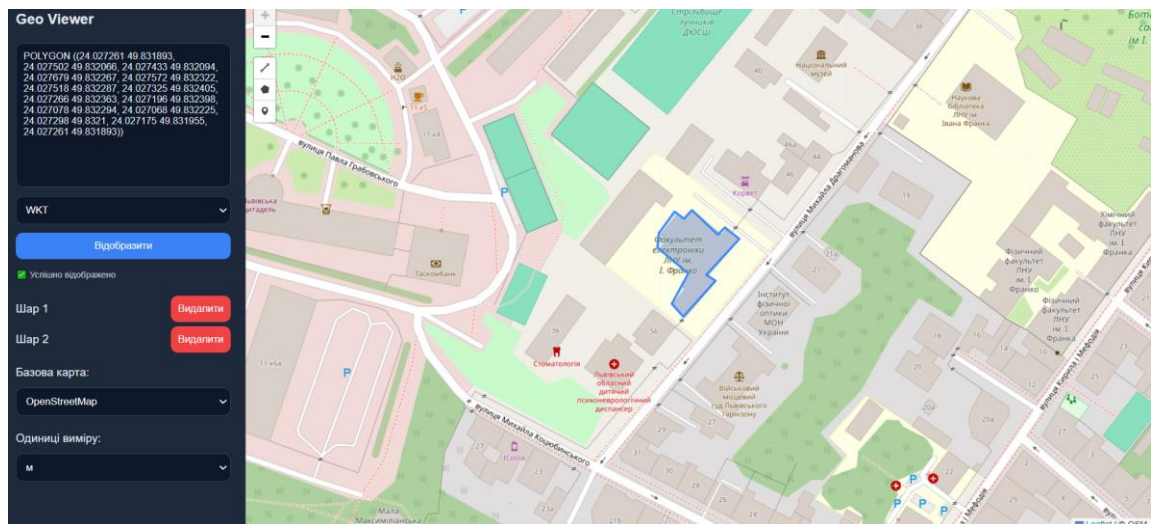


Рис. 4.3.2. Вставка тексту формату WKT та його відображення

Для ліній та полігонів додатково реалізовано обчислення довжини та площі за допомогою бібліотеки Turf.js, а результати виводяться у вигляді роликів при кліку на об'єкт. Також користувач може змінювати одиниці вимірювання, що впливає на відображення розрахованих значень.

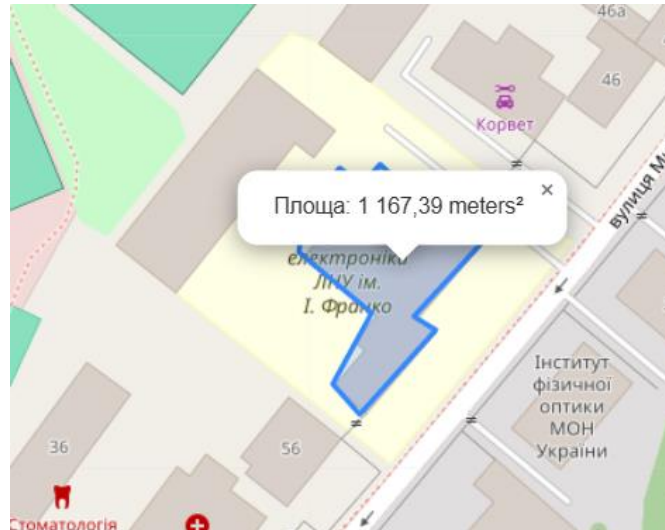


Рис. 4.3.2. Відображення площі фігури в м²

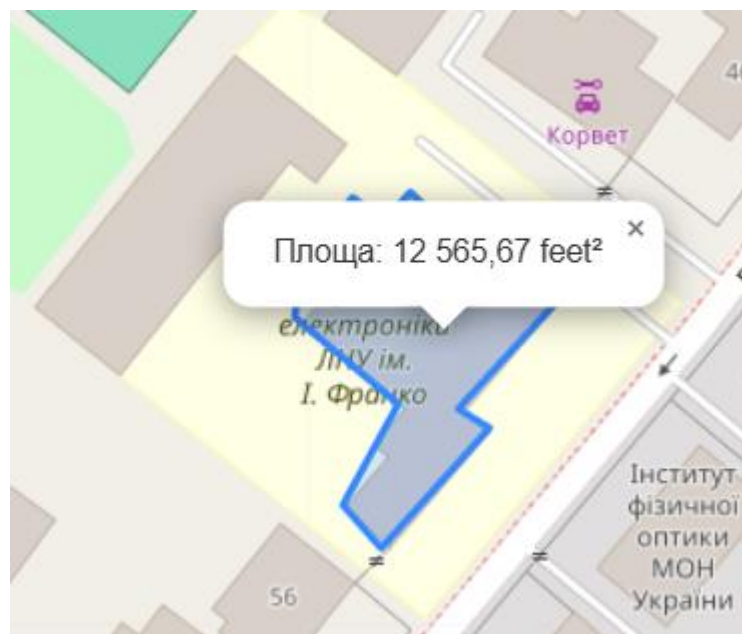


Рис. 4.3.3. Відображення площі фігури в футах

Кожен доданий об'єкт зберігається як окремий шар та відображається у списку шарів, що дозволяє керувати їхнім станом.

4.3. Робота з шарами та зміна фону карти

У застосунку реалізовано механізм роботи з шарами, який дозволяє користувачу видаляти додані геометричні об'єкти через інтерфейс списку

шарів. Кожен новий об'єкт додається як окремий шар Leaflet, що забезпечує незалежне керування ними.



Рис. 4.3.1. Можливість видалення створених шарів

Також реалізовано підтримку декількох базових картографічних підкладок, зокрема OpenStreetMap, Esri World Imagery та CartoDB Voyager. Користувач може змінювати базову карту через випадаючий список, при цьому попередній шар автоматично замінюється без перезавантаження сторінки.

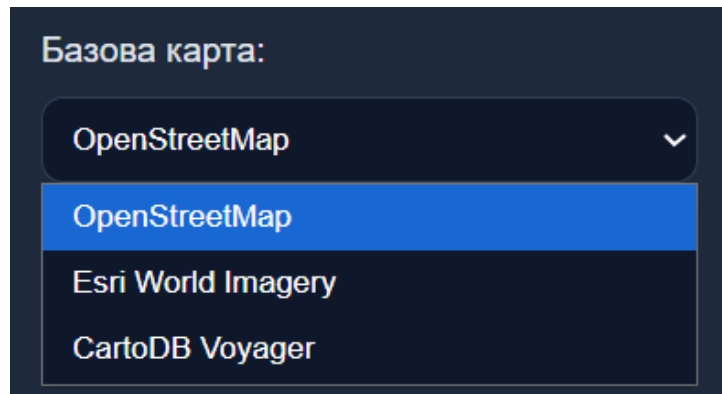


Рис. 4.3.2. Список базових карт

Додатково система підтримує динамічне оновлення інтерфейсу при зміні одиниць вимірювання, що впливає на відображення обчислених довжин та площ для відповідних геометричних об'єктів.

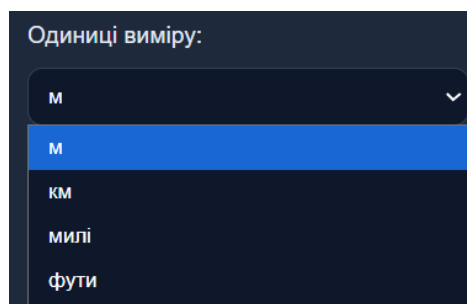


Рис. 4.3.3. Список одиниць виміру

Висновок

У ході виконання курсової роботи було розроблено веб-застосунок для відображення геопросторових даних на інтерактивній карті. Реалізована система дозволяє користувачу вводити геодані у форматах WKT та GeoJSON, після чого виконувати їх автоматичну обробку на серверній частині та візуалізувати результат у браузері.

Під час роботи було проаналізовано особливості геопросторових даних, основні типи геометричних об'єктів та поширені формати їх представлення. Окрему увагу приділено форматам WKT та GeoJSON, які використовуються для обміну геоданими між системами та веб-застосунками.

Серверна частина реалізована на основі Java та Spring Framework і забезпечує прийом, обробку та конвертацію геопросторових даних. Клієнтська частина побудована з використанням HTML, CSS та JavaScript, що забезпечує взаємодію користувача з системою та динамічне оновлення інтерфейсу без перезавантаження сторінки.

Для реалізації інтерактивної карти використано бібліотеку Leaflet, яка дозволяє відображати геометричні об'єкти у вигляді маркерів, ліній та полігонів, а також забезпечує роботу з шарами та базовими картографічними підкладками. Додатково використано бібліотеку Turf.js для виконання геометричних обчислень, зокрема визначення довжини лінійних об'єктів та площі полігонів.

Результатом роботи є функціональний веб-застосунок, який забезпечує зручний інтерфейс для роботи з геопросторовими даними, їх візуалізації та базового просторового аналізу. Розроблена система може бути розширена

шляхом додавання нових форматів даних, аналітичних функцій або інтеграції з зовнішніми геоінформаційними сервісами.

Список використаних джерел

1. Що таке веб-розробка? - визначення з техопедії - Розвиток - 2022.
[Електронний ресурс] // Розвиток. – 2022. – Режим доступу до ресурсу: <https://uk.theastrologypage.com/web-development>.
2. Що таке API? Просте пояснення для початківців [Електронний ресурс] // Dev Україна. – 2022. – Режим доступу до ресурсу: <https://dev.ua/news/chto-takoeapi-prostym-yazykom>.
3. Learn Spring Boot [Електронний ресурс] // Baeldung. – 2022. – Режим доступу до ресурсу: <https://www.baeldung.com/spring-boot>.
4. Everything You Need to Know About Web Application Architecture
[Електронний ресурс] // Topdevs. – 2021. – Режим доступу до ресурсу: <https://topdevs.org/blog/web-app-architecture-guide>.
5. Introduction to the server side [Електронний ресурс] // MDN contributors. – 2022.
– Режим доступу до ресурсу: https://developer.mozilla.org/en-US/docs/Learn/Server-side/First_steps/Introduction.
6. Yoshitaka Shiotsu. A Beginner's Guide to Back-End Development [Електронний ресурс] / Yoshitaka Shiotsu. – 2021. – Режим доступу до ресурсу: <https://www.upwork.com/resources/beginners-guide-back-enddevelopment#what-is>.
7. Oracle Java [Електронний ресурс]. – 2022. – Режим доступу до ресурсу: <https://www.oracle.com/ua/java/>

8. Java [Электронный ресурс]. – 2022. – Режим доступа до ресурсу: <https://uk.wikipedia.org/wiki/Java>.
9. Ant vs Maven vs Gradle [Электронный ресурс] // Baeldung. – 2020. – Режим доступа до ресурсу: <https://www.baeldung.com/ant-maven-gradle>.
10. Introduction to Spring Framework [Электронный ресурс] // GeeksforGeeks. – 2019. – Режим доступа до ресурсу: <https://www.geeksforgeeks.org/introductionto-spring-framework/>.
11. GeoJSON is a format for encoding a variety of geographic data structures.
[Электронный ресурс] – Режим доступа до ресурсу <https://geojson.org/>.
12. Well-Known Text (WKT). – Режим доступа до ресурсу: <https://libgeos.org/specifications/wkt/>.